COL1101

LECTURE 14

AUGUST 21, 2026

FUNCTIONAL PROGRAMMING

LAST CLASS(ES): FUNCTIONAL PROGRAMMING PARADIGM

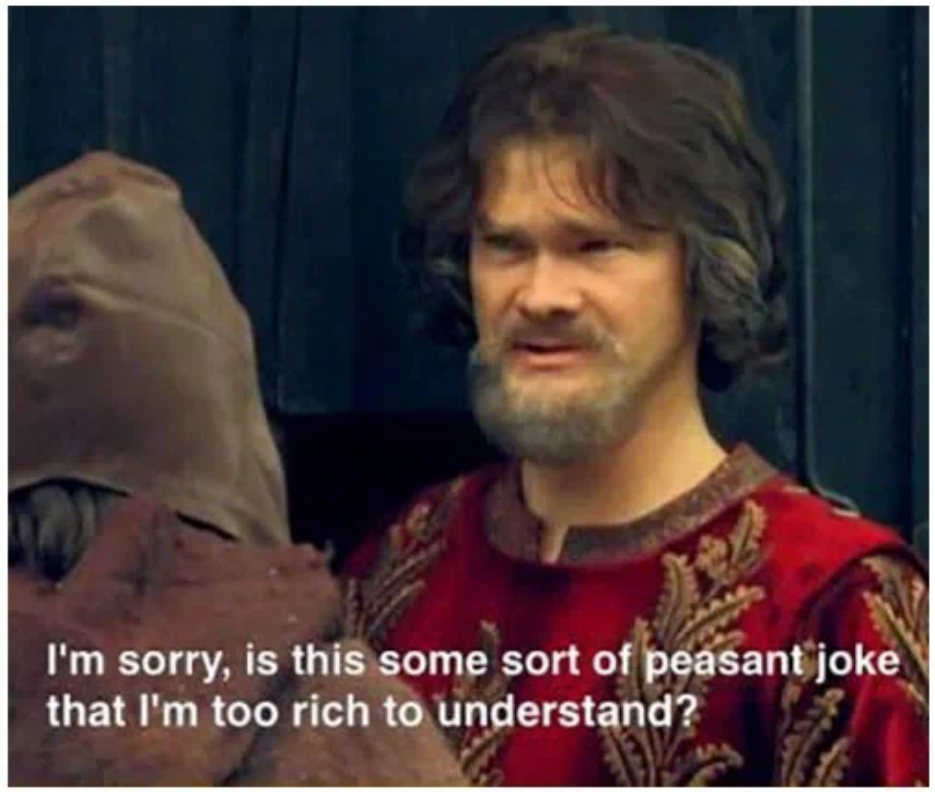
Programming paradigm

- data is immutable
- write pure functions to implement a program

FUNCTIONAL PROGRAMMING PARADIGM

Programming paradigm

- data is immutable
- write pure functions to implement a program



Ar

IDEA #1 : DATA IS IMMUTABLE

Array are in general hard to share structurally

1. Data is immutable in place

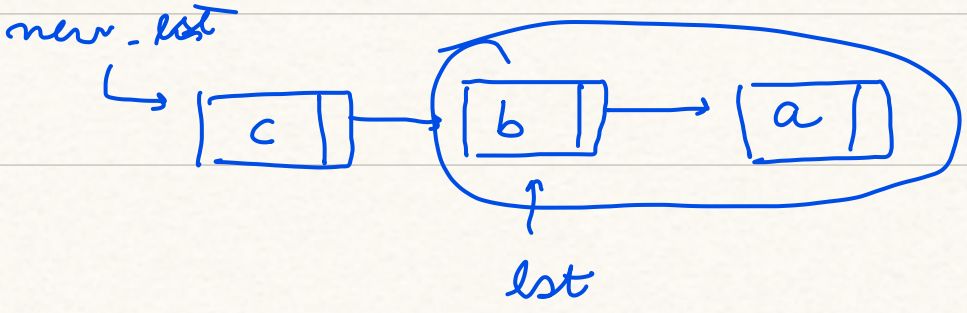
   ↳ never edit, but build a new version

   ↳ isn't that expensive? → Yes, but in some cases

   ↳ share structure whenever possible.

② structural sharing

① persistent data structures




```
lst = append(append([], 'a'), 'b')

new_lst = append(lst, 'c')
```

IDEA #2 : WRITE PURE FUNCTIONS

A pure function :

- returns same output for same input
- has no side effects (doesn't mutate external state)

IDEA #2 : WRITE PURE FUNCTIONS

A pure function :

- returns same output for same input
- has no side effects (doesn't mutate external state)

f(g(x)) → Now or 10s later

Pure expression can be replaced by its value

→ "when" and "where" it runs stops mattering

cache the output

lazy evaluation

IDEA #3: RECURSION AS PRIMARY MECHANISM FOR ITERATION

Recursion provides a natural way to repeat computation

without mutation (state in arguments & return values)

argument

```
def size(lst):
  if lst is None:
    return 0
  return 1 + size(lst.next)
```

state

size1=0 def size(lst)
t=lst
while t is not None:
    size1 += 1
    t = t.next
return size

IDEA #3: RECURSION AS PRIMARY MECHANISM FOR ITERATION

Recursion provides a natural way to repeat computation without mutation (state in arguments & return values)

```
def size(lst):
  if lst is None:
    return 0
  return 1 + size(lst.next)
```

```
def fact(n):
  return fact_tail(n, 1)

def fact_tail(n, acc):
  if n == 1:
    return acc
  else return fact_tail(n-1, n*acc)
```

def fact (n):
  if n == 0:
    return 1
  else return n × fact (n-1)

→ Tail recursion

IDEA #4 : HIGHER-ORDER FUNCTIONS

Functions can take another function as argument,

return functions, or both

↳ enable composition (implement behavior) ↳ code reuse

```
def map_(f, lst):
  if lst is None:
    return None
return append(map_(lst.next), f(lst.value))
```

$[a_0, a_1 \ldots a_n] \rightarrow [f(a_0), f(a_1) \ldots f(a_n)]$

IDEA #4 : HIGHER-ORDER FUNCTIONS

Functions can take another function as argument,

return functions, or both

↳ enable composition ↳ code reuse

```
def map_(f, lst):
  if lst is None:
    return None
  return append(map_(lst.next), f(lst.value))
```

closure

```
def adder(n):
  def add(x):
    return x + n
  return add
```

IDEA #4 : HIGHER-ORDER FUNCTIONS

Functions can take another function as argument,

return functions, or both

↳ enable composition ↳ code reuse

```
def map_(f, lst):
  if lst is None:
    return None
  return append(map_(lst.next), f(lst.value))
```

```
def adder(n):
  def add(x):
    return x + n
  return add
```

```
def compose(f, g):
  def h(x):
    return f(g(x))
  return h
```

One Last Piece of Toolkit

```
def size(lst):
    if lst is None:
        return 0
    return 1 + size(lst.next)
```

```
def total(lst):
    if lst is None:
        return 0
    return lst.value + total(lst.next)
```

```
def map_(f, lst):
    if lst is None: return None
    return append(map_(f, lst.next), f(lst.value))
```

reduce

One Last Piece of Toolkit

```
def size(lst):
  if lst is None:
    return 0
  return 1 + size(lst.next)
```

```
def total(lst):
  if lst is None:
    return 0
  return lst.value + total(lst.next)
```

```
def map_(f, lst):
  if lst is None: return None
  return append(map_(f, lst.next), f(lst.value))
```

→ Same base case

→ Else, do a little work on lst.value &

⇒ recurse on lst.next & reduce the two

REDUCE / FOLD

linked list → lst, f → combine, accumulator → acc

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

```
def combine(acc, x):
    return acc + x
```

```
def total(lst): return reduce(lst, lambda acc, x: x+acc, 0)
```

E.g. [1, 2, 3]

It 1: reduce([2, 3], combine, 1)

It 2: reduce([3], combine, 3)

It 3: reduce([], combine, 6)

It 4: return 6

Reduce / Fold

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

```
def total(lst): return reduce(lst, lambda acc, x: x+acc, 0)
```

```
def size(lst): return reduce(lst, lambda acc, x: acc + 1, 0)
```

Reduce / Fold

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

```
def total(lst): return reduce(lst, lambda acc, x: x+acc, 0)
def size(lst): return reduce(lst, lambda acc, x: acc + 1, 0)
```

```
def reverse(lst): return reduce(lst, lambda acc, x: Node(x, acc), None)
```

[1, 2, 3]

It # 1 :

combine(acc, lst.value)

reduce([2,3], , Node(1, None))

[1]

It # 2 :

reduce([3], , Node(2, [1]))

[2, 1]

It # 3 :

return → [3, 2, 1]

MAP IS ALSO A REDUCE

Recall Map- function

```
def map_(f, lst):
  if lst is None:
    return None
  return append(map_(lst.next), f(lst.value))
```

CLAIM: map- and map1 are equivalent

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

reverse (

```
def map1(lst, f): return reduce(lst, lambda acc, x: Node(f(x), acc), None)
```

[1, 2, 3] → [f(3), f(2), f(1)]

Node (x, acc)

→ [f(1), f(2), f(3)]

QUIZ: WHAT DOES THE FUNCTION COMPUTE?

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

```
def a(lst): return reduce(lst, lambda acc, x: acc + 1 if x%2 == 0 else acc, 0)
```

counting # of even elements in a list

QUIZ: WHAT DOES a COMPUTE?

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

```
def a(lst): return reduce(lst, lambda acc, x: acc + 1 if x%2 == 0 else acc, 0)
```

x None

return reduce (lst, lambda acc, x : x , None)

[1, 2, 3] It 1 reduce ([2,3] 1)

It 2 ([3] 2)

It 3 (3)

QUIZ: WRITE A REDUCE FUNCTION

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

write product-list as a single reduce.

reduce (lst , ,)

combine = lambda acc, x : acc * x

acc = 1

QUIZ: WRITE A REDUCE FUNCTION

```
def reduce(lst, combine, acc):
  if lst is None:
    return acc
  return reduce(lst.next, combine, combine(acc, lst.value))
```

Write max-list as a single reduce. Assume list is non-empty

reduce (lst , combine , acc)

combine: lambda acc, x : x if x > acc else acc

acc : $-\infty$ or lst.value

NOTE ABOUT PYTHON

```
from functools import reduce, partial     # map, filter are built in;

map(f, xs)                                # lazily apply f to each item      -> an iterat
filter(pred, xs)                          # lazily keep items where pred(x)  -> an iterat
reduce(f, xs, init)                       # left fold: f(...f(f(init, x0), x1)...)  -> on
partial(f, a)                             # pre-load an argument             -> a new fun
```

MW

map object

def adder(n):
    def add(x, n)
        return x+n
    return add

⟷ partial(adder, n)

COMPOSING FUNCTIONS

```
def compose2(f, g):
    def h(x):
        return f(g(x))
    return h
```

$[f_1, f_2, f_3 \ldots f_n]$

$f_1(f_2(f_3 \ldots f_n(x)))$

```
def compose(*funcs):
    return functools.reduce(compose2, funcs)
```

acc is optional

COMPOSING FUNCTIONS

```
def compose2(f, g):
    def h(x):
        return f(g(x))
    return h
```

```
def compose(*funcs):
    return functools.reduce(compose2, funcs)
```

LAST CLASS : `total(map_(square, filter_(is_even, lst)))`

COMPOSING FUNCTIONS

```
def compose2(f, g):
    def h(x):
        return f(g(x))
    return h
```

```
def compose(*funcs):
    return functools.reduce(compose2, funcs)
```

LAST CLASS : `total(map_(square, filter_(is_even, lst)))`

This can also be written as:

```
sum_sq_evens = compose(total, partial(map_, square), partial(filter_, is_even))
```

FUNCTIONAL PROGRAMMING VS OOP

Programmers: "object-oriented programming is really useful"

λ worshippers:



SCENARIO #1

You are to implement selection sort (Bubble sort) on an array of element

```
A[i], A[j] = A[j], A[i]
```

in-place mutation # mu

```
A = A[:i] + [A[j]] + A[i+1:j] + [A[i]] + A[j+1:]
```

→ FP

↳ O(n)

↳ O(n³)

SCENARIO #2

```
mini_icon.cart = asha_cart
checkout.cart  = asha_cart
asha_cart.add(book)

cart2 = add(asha_cart, book)
```

sharing can be bad but it can also be okay to share reference

SCENARIO #3

imperative programming

```
def greet(name):
    print(f"Hi {name}")
```

FINAL VERDICT

The question is never imperative or functional

① One way to ask:

- Is mutation essential or incidental?

FINAL VERDICT (?)

The question is never imperative or functional

① One way to ask:

- Is mutation essential or incidental?

② Also not either-or. Modern languages & programs are multi paradigm.